

Information Retrieval System - Assignment 1 Report

Course: Information Retrieval (Merged - AIMLCZG537/DSECLZG537) | Semester: S2-25

Assignment: 1 | Date: June 2026 | Deployment: <https://ir-assignment.streamlit.app/>

GitHub Repository: <https://github.com/Sankari-Masters/IRAssg1>

1. Implementation Summary

The Streamlit application implements the complete Information Retrieval workflow in seven connected modules. It loads sample or uploaded documents, preprocesses text, builds indexes, supports ranked and phrase retrieval, compares dictionary structures, and demonstrates tolerant retrieval for imperfect queries.

- Document upload: accepts TXT/CSV input or loads sample documents from the dataset folder and displays them in the UI.
- Preprocessing: applies regex tokenization, lowercasing, stop-word removal, hyphen splitting, stemming, and lemmatization with fallback logic.
- Inverted index: stores term frequencies and document frequencies, then supports term queries with ranked output.
- Phrase queries: builds biword and positional indexes; the positional index verifies exact offsets and removes biword false positives.
- Dictionary search: compares balanced BST and B-Tree lookup with nanosecond timing using `time.perf_counter_ns()`.
- Tolerant retrieval: uses k-gram wildcard matching, edit-distance spelling correction, and Soundex phonetic matching.
- Inference module: generates answers to compulsory discussion questions from the computed experimental results.

1.1 Technical Details

- `preprocess_text` returns intermediate outputs, making every preprocessing step inspectable.
- `compare_stem_and_lemma` builds separate indexes, computes cosine similarity, measures vocabulary reduction, and compares average top-3 scores.
- `build_phrase_indices` creates biword and positional indexes in one pass; false positives are `biword_results` minus `positional_results`.
- BST uses a balanced `BSTNode` dataclass, while B-Tree uses degree 8 nodes for a lower tree height.
- Wildcard search intersects k-grams and verifies matches with `fnmatch`; edit distance and Soundex handle spelling and phonetic variation.

2. Experimental Results

The dataset contains six sample text documents. The default configuration enables lowercasing, stop-word removal, hyphen handling, and lemmatization. Before normalization the vocabulary contains 169 unique terms; after lemmatization it contains 161 terms, giving a 4.7% reduction.

Stemming vs Lemmatization for query: machine learning algorithms

Method	Vocab	Reduction	Retrieved docs	Avg top-3 cosine
Stemming	145	14.2%	5	0.4436
Lemmatization	161	4.73%	5	0.4427

Conclusion: stemming is slightly better here, but the score gap is only 0.0009.

Phrase Query Processing

Query: machine learning algorithms
Biword index: 1 document retrieved, 1 false positive.
Positional index: 0 documents retrieved, 0 false positives.
Conclusion: positional indexing is more accurate because it checks exact consecutive positions.

Dictionary Search Performance

Setup: 60 queries, 145 dictionary terms, 50 existing terms and 10 missing terms.
BST height: 8; average total time: 0.000722 ms.
B-Tree height: 3; average total time: 0.001099 ms.
Conclusion: BST is faster for this small in-memory collection; B-Tree scales better for large or disk-based indexes.

Tolerant Retrieval

Wildcard query retrieval*: matched retrieval and retrieved 2 documents.
Spelling query retrieval: top suggestion is retrieval with edit distance 1.
Phonetic query lern: Soundex code L650 matches learn.
Conclusion: the model handles prefix, spelling, and phonetic variation effectively for this dataset.

3. Inferences and Discussion

- The best preprocessing combination is lowercasing, stop-word removal, hyphen handling, and stemming/lemmatization; stemming is marginally better because it reduces vocabulary more while keeping retrieval quality comparable.
- The positional index is more accurate for phrase search because it rejects documents where query terms occur as separated biword pairs instead of one exact phrase.
- BST is faster in this small in-memory experiment, but B-Tree has lower height and is preferable for large collections, disk access, and better cache locality.
- The tolerant retrieval model is effective for this dataset: wildcard matching retrieves expected terms, edit distance corrects misspellings, and Soundex groups similar-sounding terms.
- Main limitations are in-memory scale, lack of relevance-judgment evaluation, English-oriented Soundex, expensive brute-force edit distance, limited input formats, and no Boolean/proximity/query-expansion support.

- Future improvements should add BM25 or TF-IDF ranking, Boolean and proximity queries, disk-based indexing with Whoosh or Elasticsearch, relevance metrics, more parsers, stronger spelling correction, and result caching.

4. Screenshots

This section provides screenshot evidence for each implemented Streamlit module and its corresponding output.

4.1 Document Upload

Fig. 1. Document upload interface

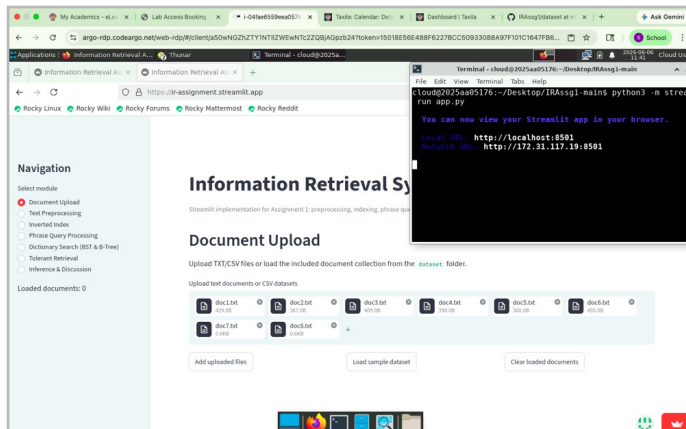
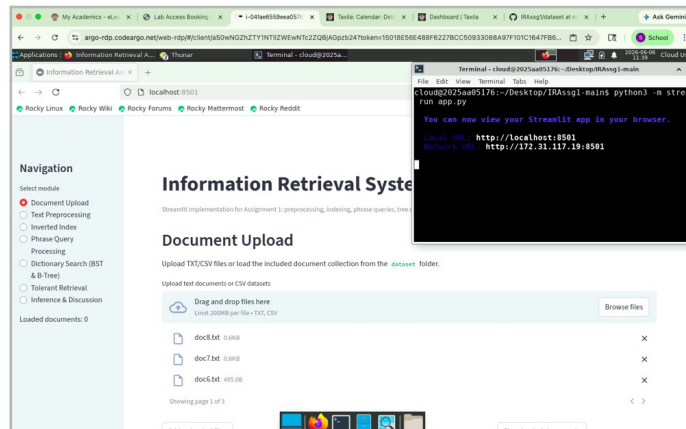


Fig. 2. Loaded sample documents



4.2 Text Preprocessing

Fig. 3. Preprocessing configuration

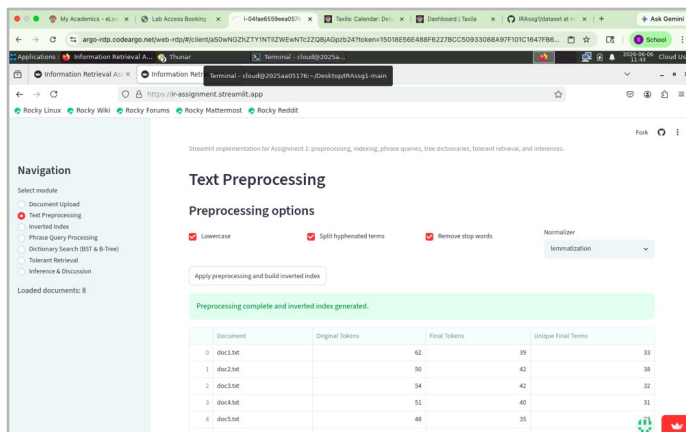


Fig. 5. Sample preprocessing output

Fig. 4. Preprocessing options panel

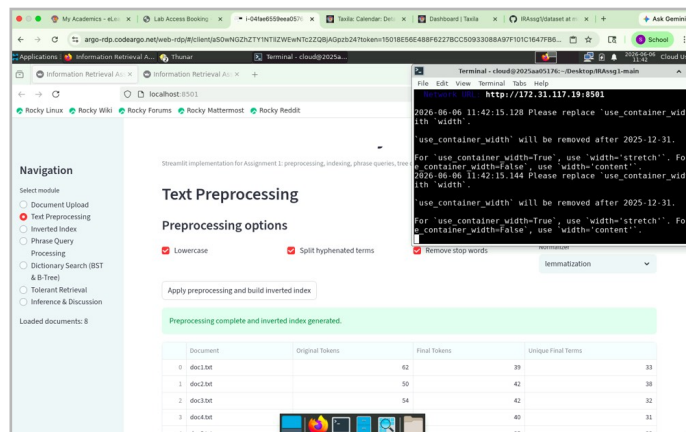


Fig. 6. Step-wise preprocessing result

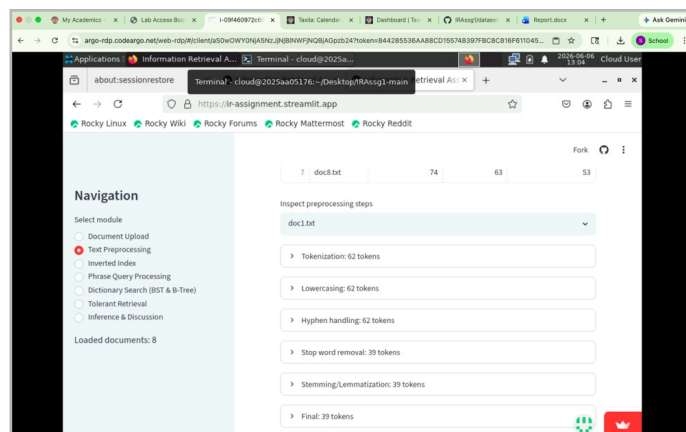
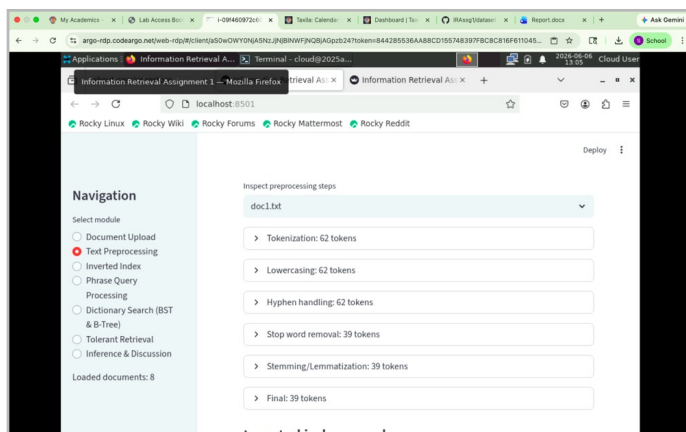


Fig. 7. Stemming vs lemmatization comparison

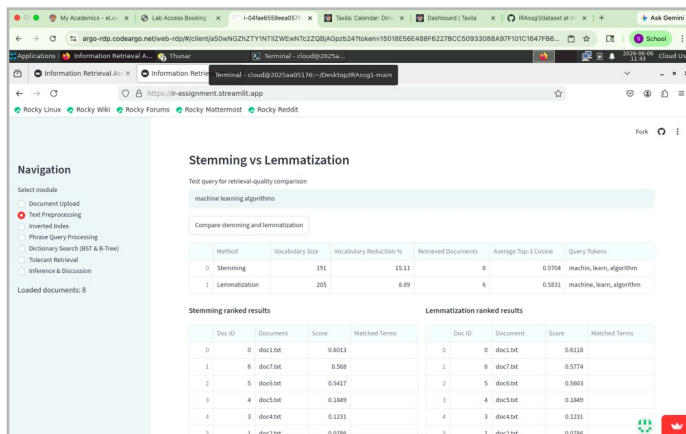
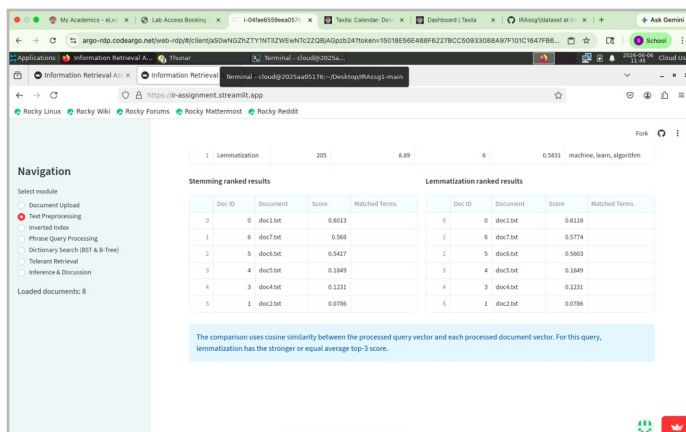


Fig. 9. Comparison table continuation



4.3 Inverted Index

Fig. 11. Built inverted index

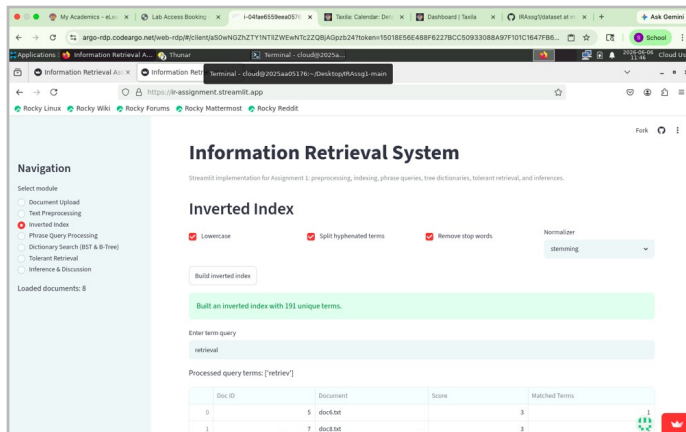
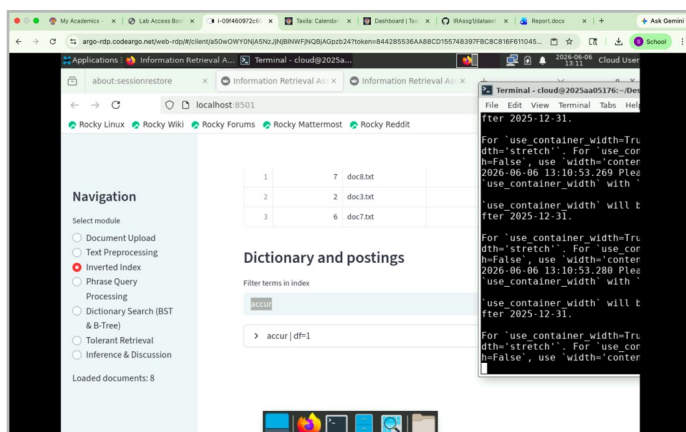


Fig. 13. Term query results



4.4 Phrase Query Processing

Fig. 8. Vocabulary and retrieval metrics

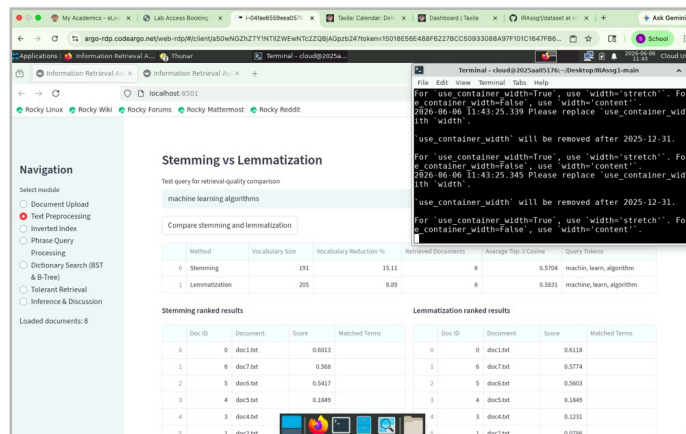


Fig. 10. Normalization comparison output

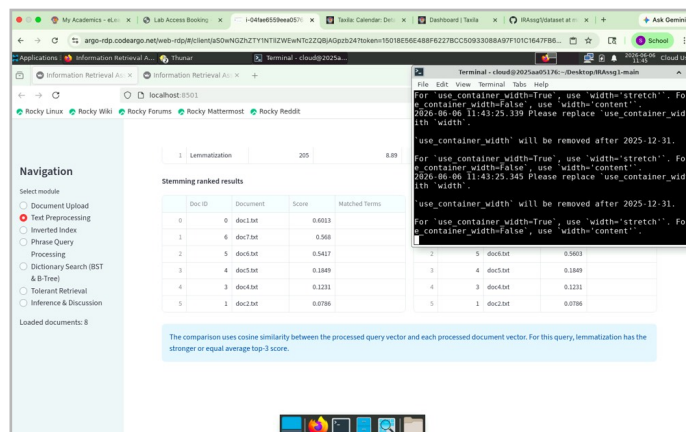


Fig. 12. Inverted index details

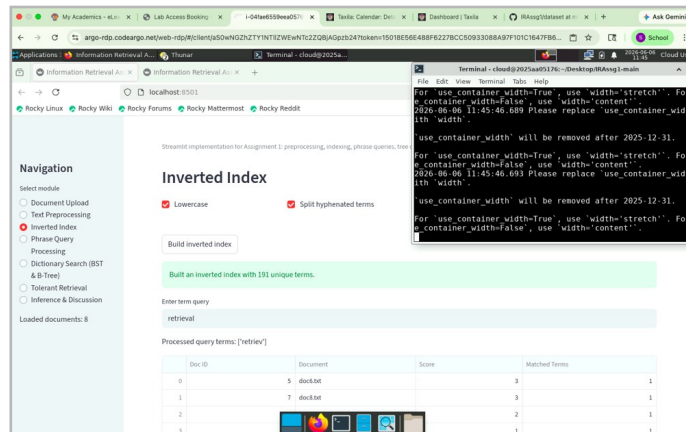


Fig. 14. Ranked retrieval output

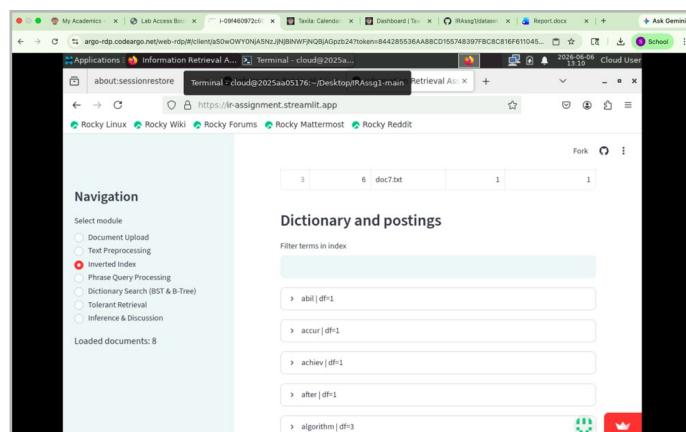


Fig. 15. Biword index sample

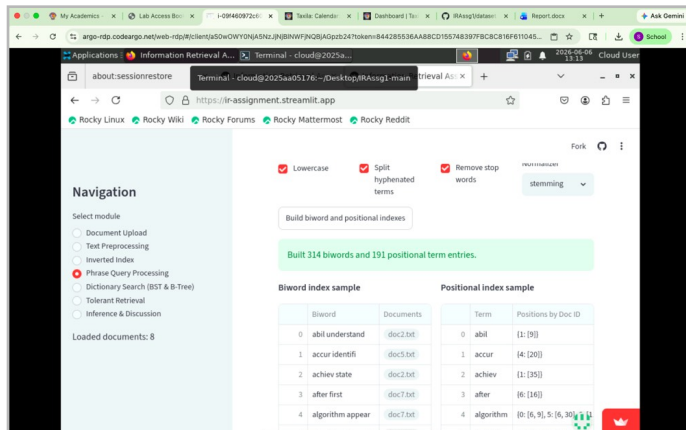


Fig. 17. Positional index sample

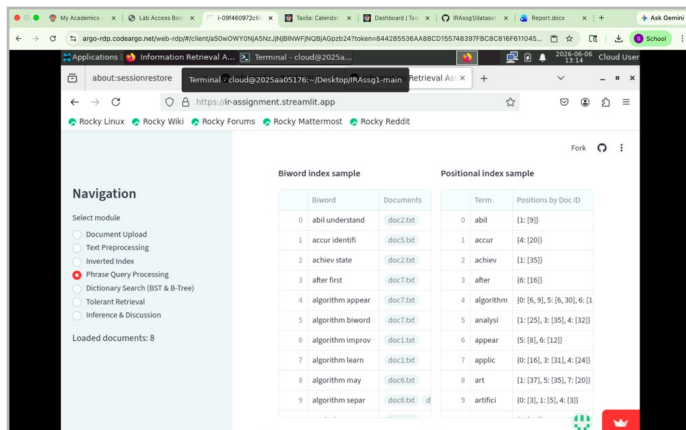
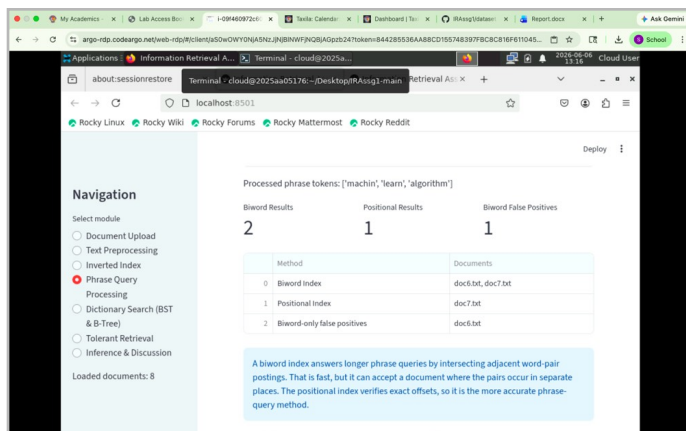


Fig. 19. False-positive analysis



4.5 Dictionary Search

Fig. 20. BST and B-Tree heights

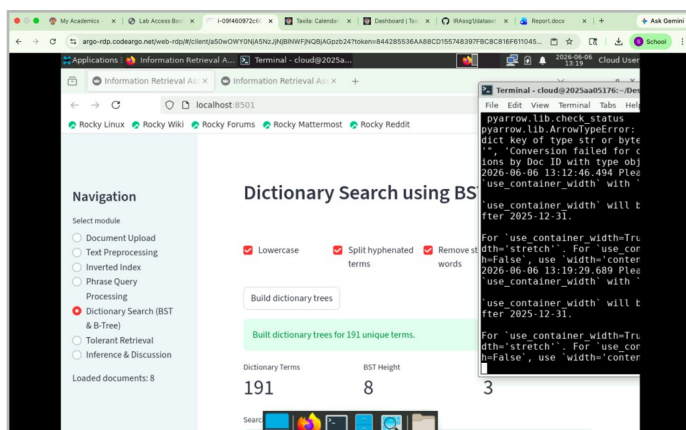


Fig. 16. Biword postings output

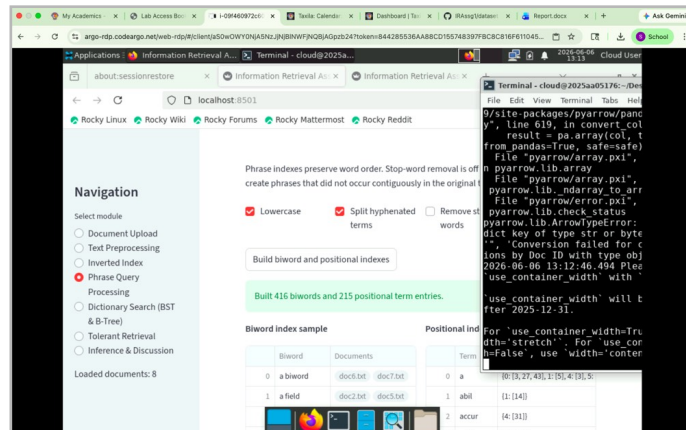


Fig. 18. Phrase query result

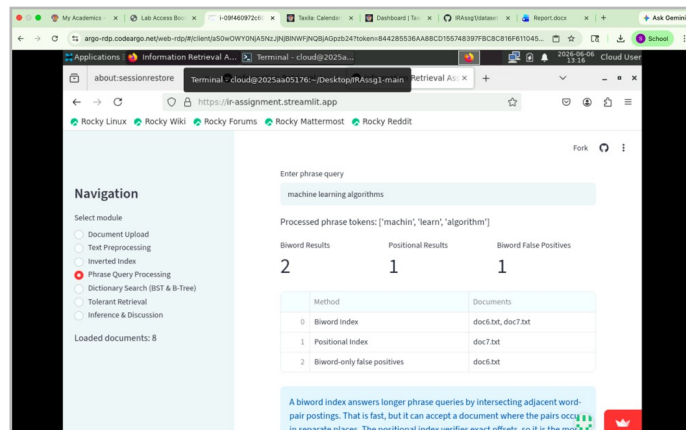


Fig. 21. Tree comparison setup



Fig. 22. Single query comparison

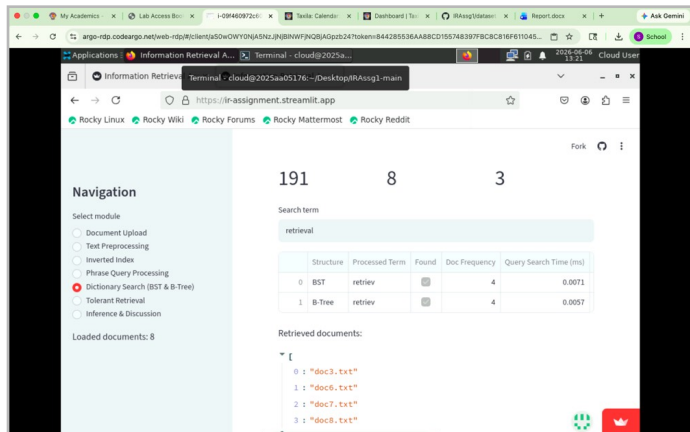
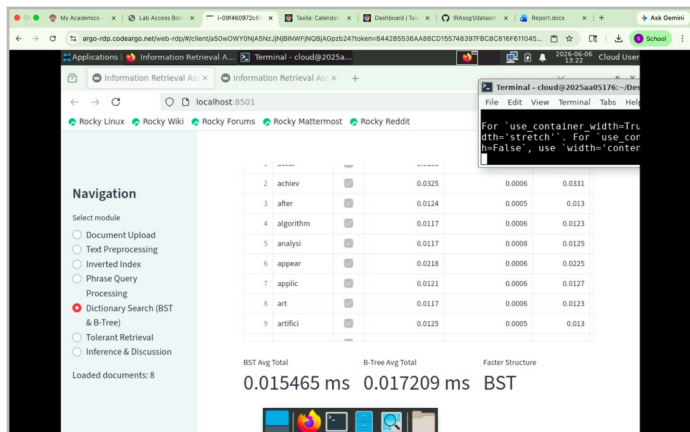


Fig. 24. Experimental result table



4.6 Tolerant Retrieval

Fig. 26. Wildcard query result

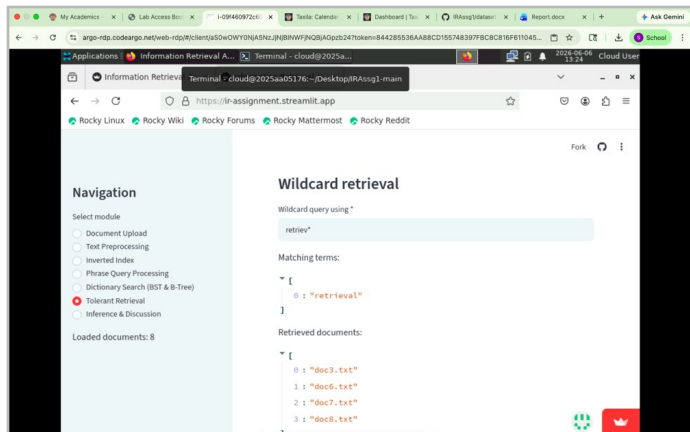


Fig. 28. Edit-distance suggestions

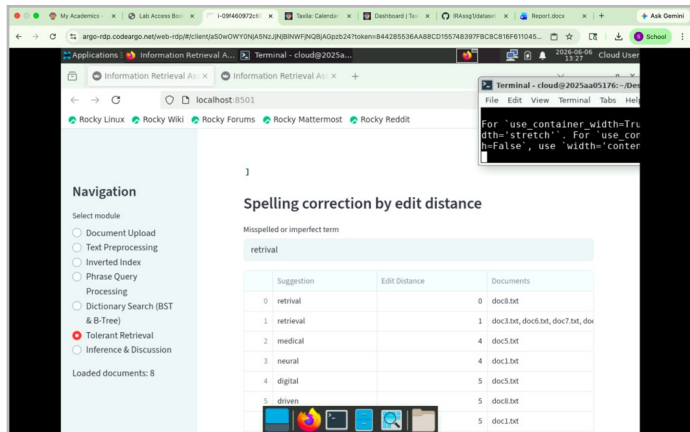


Fig. 23. Dictionary search timing

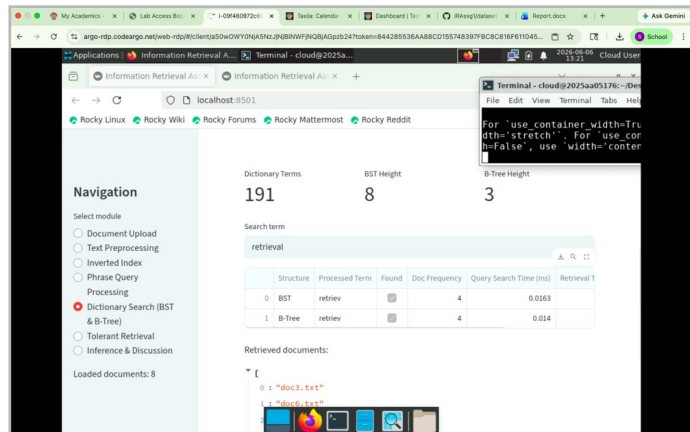


Fig. 25. BST vs B-Tree summary

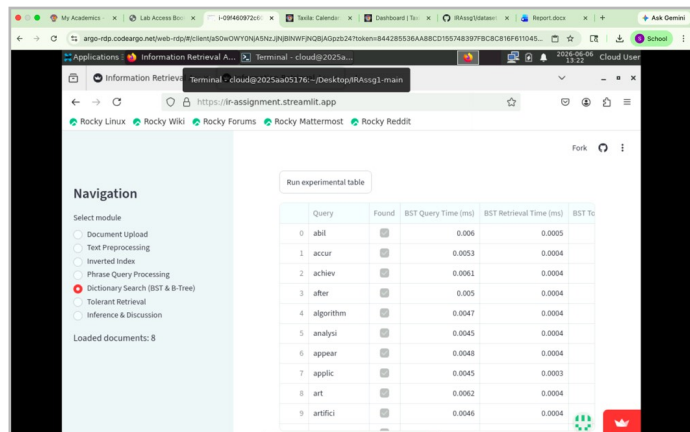


Fig. 27. Wildcard matching terms

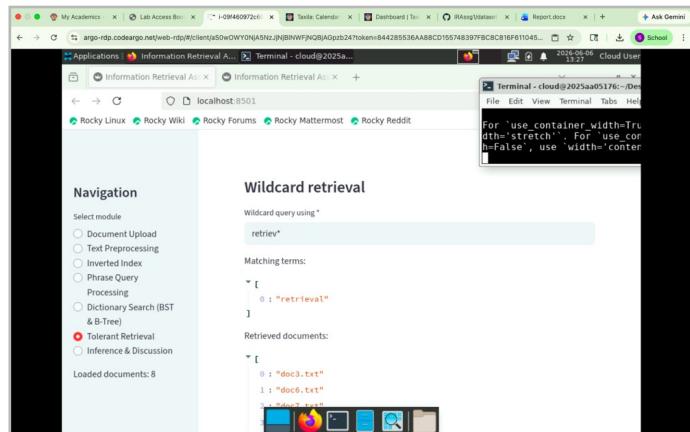


Fig. 29. Spelling correction output

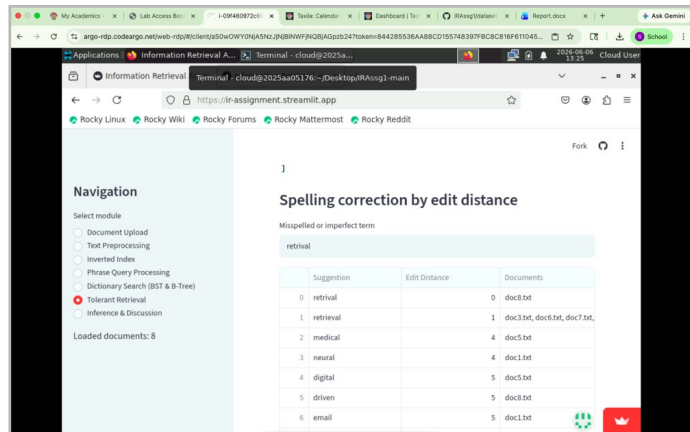
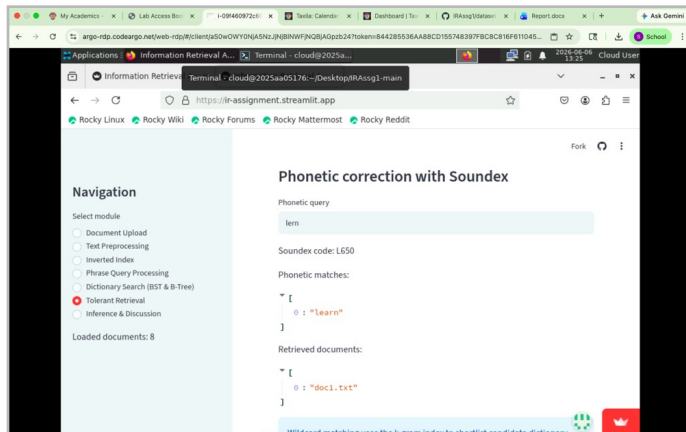


Fig. 30. Soundex phonetic result



4.7 Inference and Discussion

Fig. 32. Dynamic inference answers

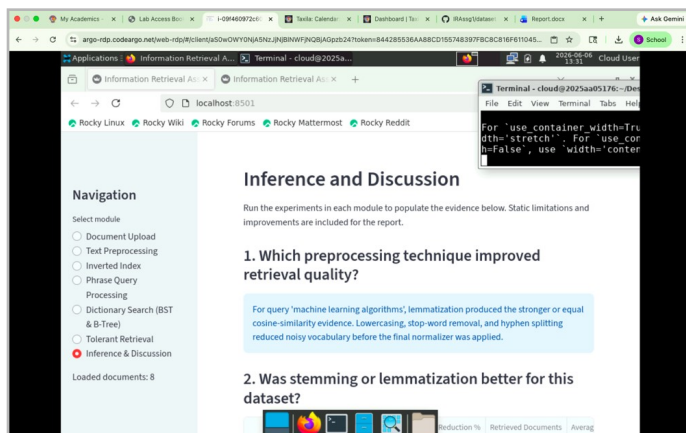
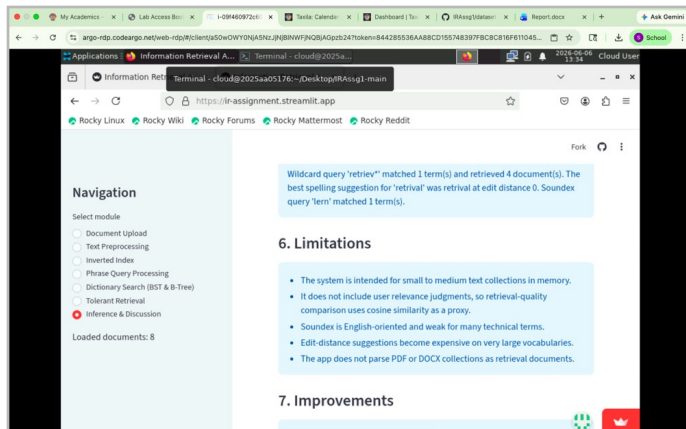


Fig. 34. System limitations



5. Conclusion

This assignment implements an end-to-end IR system with preprocessing, inverted indexing, phrase retrieval, dictionary search, and tolerant retrieval. The experiments show that stemming and lemmatization perform similarly, positional indexes are more precise than biword indexes for exact phrase queries, BST is faster for this small in-memory dataset, and B-Tree is better suited to larger disk-based collections. The deployed Streamlit workflow demonstrates all required components and presents dynamic inferences from the observed results.

6. References

- Manning, C. D., Raghavan, P., and Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
- Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130-137.
- Odell, M. K., and Russell, R. C. (1918). Soundex: A coding system for indexing names. US Patent.
- NLTK Project. (2025). Natural Language Toolkit. <https://www.nltk.org/>
- Streamlit Documentation. (2025). <https://docs.streamlit.io/>
- Project repository: <https://github.com/Sankari-Masters/IRAssg1>

Fig. 31. Phonetic match output

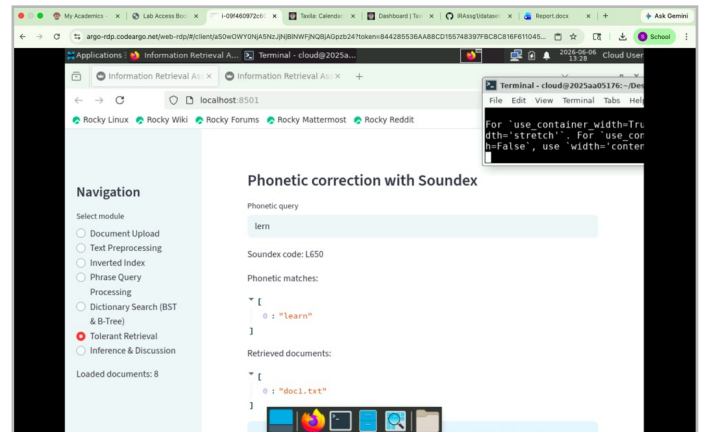


Fig. 33. Inference discussion output

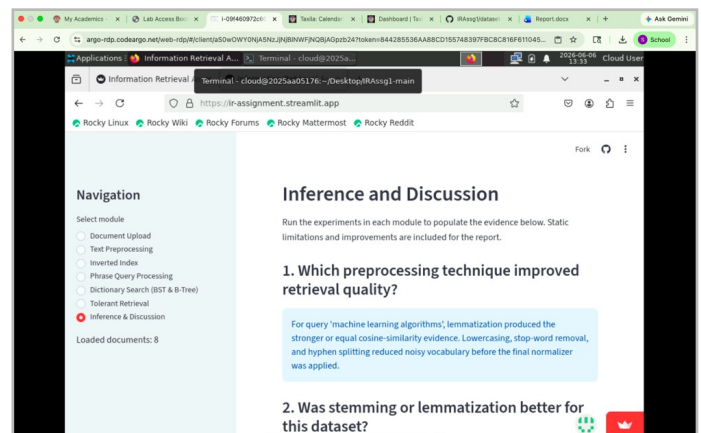


Fig. 35. Improvement suggestions

